of the files; it returns a URL configuration that can be used to serve files in `DEBUG` mode. Although your uploaded files aren't static, in development, you can pretend they are and essentially add them to the list of things the development server serves up.

This is a little bit "hackish," but the only alternative is to run your own web server when doing development, and that's a lot of extra work. In production, the web server will take care of this, so you need to make sure you aren't accidentally running both mechanisms. To do this, wrap the code in a check for `DEBUG == True`. Open RiffMates/RiffMates/urls.py, and make the changes shown in the following listing.

Listing 7.15 Serve uploaded files as if they were static files, in development mode

```
# RiffMates/RiffMates/urls.py
...
from django.conf import settings
from django.conf.urls.static import static
...
if settings.DEBUG:
    urlpatterns += static(settings.MEDIA_URL,
        document_root=settings.MEDIA_ROOT)
```

Allows access to values configured in settings.py

The static() function that registers static serving URLs

Only serve uploaded files this way if you are in DEBUG=True mode. This code goes below your urlpatterns list.

Add the new route returned by static() into urlpatterns.

Before moving on to handling uploaded files, you should test your configuration. You can do this by copying any file into the uploads directory. For example, copy the RiffMates logo into ../outside/RiffMates/uploads. With the development server running, visit http://localhost:8000/media/logo.png to verify your upload directory is working. Don't forget to remove the file when you're done.

7.4.2 Storing and referencing uploaded files

To allow a user to upload a file, you need a form and a view. Before doing that, though, let's start by creating the associated model changes and using the Django Admin to verify that it works.

You store a reference to an uploaded file in the database using either a `FileField` or an `ImageField`. Both hold a path to the file on the server, while the `ImageField` also validates that the uploaded file is an image. The `ImageField` uses the Pillow library to process the uploaded file, so a valid image is any format that Pillow handles. This includes the most common bitmap based image formats found on the web, including GIF, JPG, PNG, and WebP. Note that Pillow does not support vector-based image formats—SVG files cannot be stored in an `ImageField`. If you're playing with SVG files, you'll have to use the more generic `FileField` instead.

In chapter 4, you added the `Venue` model to store information about the clubs hosting bands. At the moment, the venue listing page only has the names of the venues; let's extend that, so each venue can have a description and an optional image. The first step in this direction is to add fields to the `Venue` model object: a `TextField`